Lecture 3

Vision is about interpreting the image holistically in terms of how thins move and change temporally.

Facial detection: one of the original and most impactful computer vision applications

What computers see?

To a computer, images are just numbers.

- In a grayscale image: every pixel is represented by one number, as a two-dimensional array of one number

- In a color image: every pixel is represented by three numbers (RGB), as a three-dimensional array

Task in Computer Vision

Regression allows us to output a continuous value.

Classification allows us to output a discrete class label.

Manual Feature Extraction

Our models must be invariant to these changes to be robust. Usually, the detection of these features is where traditional machine learning breaks down and deep learning kicks in. In the machine learning world, defining features is a manual human process. In deep learning, we define these features using data.

Hierarchy of features

The depth of the model curates the hierarchy, with every layer adding a level of complexity or expressivity

Why fully connected networks fail for images

If we applied this multi-layer network to computer vision, we would have a problem.

To feed an image into a fully connected network, we have to flatten it into a one-dimensional input. We destroy the spatial structure by flattening it.

This creates a mssive number of parameters because every single pixel is connected to every single hidden neuron.

> This results in an extremely expensive model

Then how can we preserve this spatial structure ?

We keep the two-dimensional structure, but only connect inputs to output neurons that are locally connected

One neuron only sees inputs from a local patch of pixels.

Each patch gives rise to one output neuron, and as the patch shifts, it defines the next neuron

How can we weight the patch to detect particular features?: Convolution

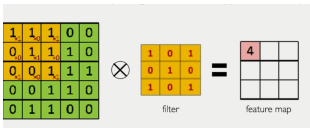
1. apply a set of weights

2. use multiple filters to extract different features

3. spatially share parameters of each filter

How does this extract features?

The convolution operation



We slide the 3 by 3 filter over the input image, element-wise multiply and add the outputs

CNNs for Classification

1. Convolution: Apply filters to generate feature maps

2. Non-linearity: Often ReLU

3. Pooling: Downsampling operation on each feature map

Train model with image data. Learn weights of filters in convolutional layers.

Convolutional Layers: Local Connectivity

For a neuron in hidden layer:

- Take inputs from patch

- Compute weighted sum

- Apply bias

4x4 filter: matrix of weights w_{ij}

$$\sum_{i=1}^{4}\sum_{j=1}^{4} w_{ij}\, x_{i+p,j+q} + b$$

for neuron (p,q) in hidden layer

1) applying a window of weights

2) computing linear combinations

3) activating with non-linear function

CNNs: Spatial Arrangement of Output Volume

Layer Dimenstions: h x w x d

Where h and w are spatial dimensions, d (depth) is number of filters

Stride: Filter step size

Receptive Field: Locations in input image that anode is path connected to

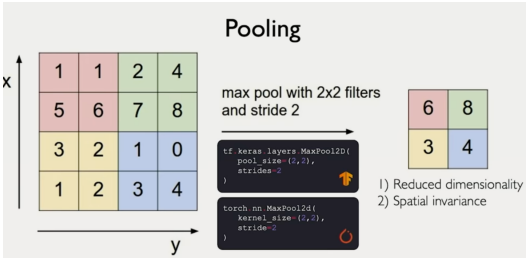
Introducing Non-Linearity

- Apply after every convolution operation

- ReLU (Rectified Linear Unit): pixel-by-pixel operation that replaces all negative values by zero.

>> Non-linear operation

Pooling



Then how else can we downsample and preserve spatial invariance?

> Representation Learning in Deep CNNs

Low level features (Edges, dark spots)

>

Mide level features (Eyes, ears, nose)

>

High level features (Facial structure)

CNNs for Classification:

1. Features Learning

2. Probabilities

Object detection with R-CNNs

R-CNNs algorithm: find regions that we think have objects. Ise CNN to classify

Input image > Extract region proposals > Compute CNN features > Classify regions

Problems – Slow. Many regions; time intensive inference.

- Brittle. Manually defined region proposals

Faster R-CNN learns region proposals

End-to-End Framework for autonomous navigation

Entire model is trained end-to-end without any human labelling or annotation